

Word Count in Python

A Straightforward Script vs. a MapReduce-Style Rewrite

This document contains two working Python programs that both count word frequencies from a small text file, plus a plain-language explanation of the MapReduce pattern the second program is modeled on. The first script reads, cleans, and counts words in one straight pass. The second restructures the exact same problem into three distinct phases — Map, Shuffle & Sort, and Reduce — mirroring how large-scale distributed data systems like Hadoop and Spark process data.

Script	Approach	Best for
1	Single-pass, dictionary counting	Small scripts, quick tasks
2	Map → Shuffle & Sort → Reduce	Understanding distributed computing

1. Script One — Straightforward Word Count

Writes a small text file, appends a line to it, reads it back, lowercases and strips punctuation, splits it into words, tallies counts in a dictionary, sorts by frequency, and writes the results to output.txt.

script1.py

```
import string

# =====
# 1. WRITE DATA TO input.txt
# =====

with open("input.txt", "w") as file:
    file.write("Python is easy\n")
    file.write("Python is powerful\n")
    file.write("I am learning Big Data\n")
    file.write("Python is easy to learn\n")

# =====
# 2. APPEND NEW DATA
# =====

with open("input.txt", "a") as file:
    file.write("Spark is powerful\n")

# =====
# 3. READ DATA FROM input.txt
# =====

with open("input.txt", "r") as file:
    text = file.read()

print("===== ORIGINAL TEXT =====")
print(text)

# =====
# 4. CLEAN THE TEXT
# =====

# Convert to lowercase
text = text.lower()

# Remove punctuation
text = text.translate(
    str.maketrans("", "", string.punctuation)
)

# =====
```

```
# 5. SPLIT TEXT INTO WORDS
# =====

words = text.split()

# =====
# 6. COUNT WORDS
# =====

word_count = {}

for word in words:

    if word in word_count:
        word_count[word] += 1
    else:
        word_count[word] = 1

# =====
# 7. SORT WORDS BY HIGHEST COUNT
# =====

sorted_words = sorted(
    word_count.items(),
    key=lambda item: item[1],
    reverse=True
)

# =====
# 8. PRINT RESULTS
# =====

print("===== WORD COUNT =====")

for word, count in sorted_words:
    print(word, ":", count)

# =====
# 9. WRITE RESULTS TO output.txt
# =====

with open("output.txt", "w") as file:

    for word, count in sorted_words:
        file.write(f"{word}: {count}\n")

print("\nWord count saved to output.txt")
```

This is a classic single-machine approach: everything happens in one continuous pass over the data, in memory, on one process.

2. Script Two — MapReduce-Style Word Count

Same input data and same final result, but the counting logic is split into three explicit functions — `mapper()`, `shuffle_and_sort()`, and `reducer()` — so each phase of the MapReduce pattern is visible and testable on its own.

script2.py

```
import string

# =====
# 1. WRITE DATA TO input.txt
# =====
with open("input.txt", "w") as file:
    file.write("Python is easy\n")
    file.write("Python is powerful\n")
    file.write("I am learning Big Data\n")
    file.write("Python is easy to learn\n")
# =====
# 2. APPEND NEW DATA
# =====
with open("input.txt", "a") as file:
    file.write("Spark is powerful\n")
# =====
# 3. MAPPER
# =====
def mapper(text):
    # Convert text to lowercase
    text = text.lower()
    # Remove punctuation
    text = text.translate(
        str.maketrans("", "", string.punctuation)
    )
    # Split text into words
    words = text.split()
    # Emit (word, 1)
    mapped_data = []
    for word in words:
        mapped_data.append((word, 1))
    return mapped_data
# =====
# 4. SHUFFLE AND SORT
# =====
def shuffle_and_sort(mapped_data):
    grouped_data = {}
    for word, count in mapped_data:
        if word not in grouped_data:
            grouped_data[word] = []
        grouped_data[word].append(count)
    return grouped_data
# =====
# 5. REDUCER
# =====
def reducer(grouped_data):
    reduced_data = {}
```

```

        for word, counts in grouped_data.items():
            total_count = sum(counts)
            reduced_data[word] = total_count
        return reduced_data

# =====
# 6. READ INPUT FILE
# =====
with open("input.txt", "r") as file:
    text = file.read()
print("===== ORIGINAL TEXT =====")
print(text)
# =====
# 7. MAP PHASE
# =====
mapped_data = mapper(text)
print("===== MAP OUTPUT =====")
for item in mapped_data:
    print(item)
# =====
# 8. SHUFFLE AND SORT PHASE
# =====
grouped_data = shuffle_and_sort(mapped_data)
print("\n===== SHUFFLE AND SORT =====")
for word, counts in grouped_data.items():
    print(word, ":", counts)
# =====
# 9. REDUCE PHASE
# =====
reduced_data = reducer(grouped_data)
# =====
# 10. SORT RESULTS
# =====
sorted_words = sorted(
    reduced_data.items(),
    key=lambda item: item[1],
    reverse=True
)
# =====
# 11. PRINT FINAL RESULTS
# =====
print("\n===== FINAL WORD COUNT =====")
for word, count in sorted_words:
    print(word, ":", count)
# =====
# 12. WRITE RESULTS TO output.txt
# =====
with open("output.txt", "w") as file:
    for word, count in sorted_words:
        file.write(f"{word}: {count}\n")
print("\nWord count saved to output.txt")

```

Even though this still runs on a single machine, the structure is exactly what a real distributed engine (Hadoop MapReduce, Spark) would execute in parallel across many machines.

3. What Is MapReduce?

MapReduce is a programming model for processing very large datasets by splitting the work into small, independent tasks that can run in parallel across many machines, then combining their results. It was popularized by Google and later implemented in open-source tools such as Apache Hadoop and, in a more flexible form, Apache Spark.

The Three Phases

Map	Each input record is processed independently and turned into key–value pairs. In the word count example, every word in the text becomes a pair like ("python", 1). Because each record is handled on its own, this step can be split across many machines at once with no coordination needed between them.
Shuffle & Sort	All the key–value pairs produced by the Map phase are regrouped so that every value belonging to the same key ends up together. All the 1s for "python" are collected into one list, all the 1s for "powerful" into another, and so on. In a real cluster, this is the step that involves moving data between machines over the network.
Reduce	Each group of values sharing a key is combined into a single result, typically by summing, counting, or otherwise aggregating them. Each list of 1s is summed into a total count per word, producing the final {word: count} results.

Why It Matters

The point of separating a problem into Map, Shuffle & Sort, and Reduce is scalability. A single machine can only hold and process so much data. If the text file were a petabyte of web pages instead of five short sentences, the Map step could run on thousands of machines simultaneously, each handling a slice of the data, and the Reduce step could likewise be split across many machines by key. The programmer only has to write the mapper and reducer logic; the underlying system (Hadoop, Spark, etc.) takes care of distributing the data, running tasks in parallel, handling machine failures, and shuffling results between nodes.

Connecting It Back to the Scripts

- Script 1 does the whole job in one pass — fine for a small file, but it does not show how the work could be divided up.
- Script 2's `mapper ( )` function is exactly what would run, in parallel, on each chunk of data in a real cluster.
- Its `shuffle_and_sort ( )` function mimics the regrouping step that a framework like Hadoop performs automatically between the Map and Reduce stages.
- Its `reducer ( )` function is exactly what would run, in parallel, once per unique key, to produce the final totals.

In short: MapReduce turns "count everything on one machine" into "count each piece anywhere, then add the pieces together" — a pattern that scales from a five-line text file to the entire web.